

אני אחפש עבורך מידע על אתרים לזיהוי זן אתרוג ובדיקת הכשרות שלהם.

מהמידע שמצאתי, אין נראה שקיים אתר או אפליקציה ייעודית לזיהוי זן אתרוג ולבדיקת כשרותו באופן ויזואלי אוטומטי. עם זאת, אני יכול לסכם עבורך את המידע החשוב על זני האתרוגים וכשרותם:

זני האתרוגים העיקריים וכשרותם:

זנים כשרים עיקריים:

****1. אתרוג תימני:****

אתרוג תימני הוא זן שהובא ארצה על ידי עולי תימן. הוא מתאפיין בגודלו, בכך שאין לו פיטם ונחשב מהודר ובלתי מורכב. המאפיין העיקרי שלו הוא העדרם של בקבוקוני המיץ בפלחיו.

****2. אתרוג ברוורמן (מהרי"ל דיסקין):****

אתרוגים אלו גודלו בחצר ביתם של משפחת ברוורמן בפתח תקווה בתחילת המאה ה-20, והם נחשבים בחוגי המהדרים במצוות, ומועדפים במיוחד בחוגי "בריסק".

****3. אתרוג מרוקאי:****

האתרוג המרוקאי גדל באזור מרוחק על הרי אנטי אטלס שבמרוקו. פרופסור אליעזר גולדשמידט מהאוניברסיטה העברית ביקר במקום ביחד עם משלחת רבנים, בציווי הפוסק הרב יוסף שלום אלישיב, ואישר שהמקום נקי מחשש הרכבה.

איך לזהות אתרוג כשר:

****סימנים לאתרוג מורכב (פסול):****

הסימן החד-משמעי מתקבל בפתיחת האתרוג: אם מצאתם בו בקוקוני מיץ כמו שיש בפרי הלימון ובפירות ההדר, דעו בוודאות שאתרוגכם לימון הוא.

האתרוגים המורכבים של המינים קטנים בהרבה, נוטים לצבע הצהוב כצבע הלימון, ויש להם חתיכת עץ בולטת בדדו של האתרוג, ואלה מורכבים כמעט בוודאות.

****סימנים לאתרוג תימני כשר:****

הוא גדול בהרבה, במגוון צורות וגדלים, נוטה לצבעי הירוק הבהיר, ואין לו חתיכת עץ בולטת בדדו של האתרוג.

פסולים נוספים באתרוג:

אתרוג חסר פסול. למעשה יש להחמיר בכך, ואפילו חיסרון כלשהו פוסל.

בחוטמו של אתרוג, שינוי מראה פוסל במשהו. בשאר האתרוג, שינוי מראה פוסל כשהוא קיים

המלצה:

מאחר שאין אתר או אפליקציה ספציפית לזיהוי זן אתרוג, הדרך הטובה ביותר היא להסתמך על סוחרים מוכרים עם הכשרה מהימנה, או להתייעץ עם רב מקומי לגבי האתרוג הספציפי שברשותך.

יופי, זה פרויקט מרתק! בניית מערכת זיהוי ויזואלי אוטומטית לזני אתרוגים וכשרותם באמצעות בינה מלאכותית ועיבוד תמונה. כדי שאבצע מחקר יסודי שיתן לך דרכי פעולה מעשיות, יש לי כמה שאלות מבהירות:

1. ****היקף המערכת****: האם אתה רוצה לפתח אפליקציה ניידת שתזהה אתרוגים בזמן אמת, או מערכת ווב שמקבלת תמונות מועלות?
1. ****רמת הדיוק הנדרשת****: האם המטרה היא זיהוי בסיסי (תימני/לא תימני, מורכב/לא מורכב) או זיהוי מפורט יותר של זנים ספציפיים (ברוורמן, מרוקאי, חזון איש וכו') וכל סוגי הפסולים ההלכתיים?
1. ****משאבים זמינים****: האם יש לך גישה למאגר תמונות קיים של אתרוגים מסוגים, או שצריך לבנות אחד מאפס?

אתחיל כעת במחקר מעמיק על הטכנולוגיות, האלגוריתמים, ודרכי המימוש הטובות ביותר לפרויקט כזה!

מושלם! עכשיו אני מבין בדיוק מה אתה רוצה. אני אצא למחקר ממוקד על איך לבנות מערכת שתדע לזהות:

- האם האתרוג פסול או כשר
- נוכחות נקודות שחורות בשליש העליון
- זיהוי זן ספציפי (תימני, ברוורמן, מרוקאי וכו')
- הערכה אם הוא ראוי לברכה

אתחיל במחקר מעמיק על הטכנולוגיות והמתודולוגיות הספציפיות לפרויקט הזה, כולל חיפוש אחר מאגרי תמונות קיימים של אתרוגים ברשת שיוכלו לשמש בסיס לאימון המערכת!

Your AI-powered etrog identification app research report is ready. It covers existing etrog image databases, advanced fruit identification AI technologies, halachic requirements, recommended development platforms, implementation challenges, and cost analysis. The report estimates \$150,000-\$300,000 development costs over 12-18 months, with detailed technical specifications and a phased implementation strategy accounting for seasonal data collection and religious community adoption.

יש לי עבורך כמה שאלות מבהירות כדי שאוכל לעשות מחקר ממוקד יותר על היבטים ספציפיים:

1. ****צורת הזיהוי****: האם אתה מתכנן מערכת שתזהה אתרוגים בזמן אמת (כמו סריקה חיה במצלמה), או מערכת שמעלים אליה תמונה של אתרוג לניתוח?
 1. ****היקף הבעיות הטכניות****: האם יש נושאים טכניים ספציפיים שמעניינים אותך במיוחד - לדוגמה, איך לפתור בעיות תאורה, איך להתמודד עם זוויות צילום שונות, או איך להבטיח דיוק גבוה בהבחנה בין זנים דומים?
 1. ****שלב הפיתוח****: האם אתה מעוניין להתחיל עם פרוטוטיפ בסיסי (MVP) שמזהה רק את העיקר (כשר/פסול), או לקפוץ ישר לפיתוח מערכת מקיפה עם כל התכונות?
- בהתבסס על התשובות שלך, אני מעמיק במחקר על הפתרונות הטכניים הספציפיים שיעזרו לך הכי הרבה!

מושלם! עכשיו אני מבין בדיוק את החזון שלך. אתה רוצה להתחיל עם זיהוי לייב (real-time) שיספק איבחון מדויק, ואחר כך להוסיף תכונות נוספות בהדרגה.

אני אתחיל כעת במחקר מפורט על:

1. ****טכנולוגיות זיהוי לייב**** - איך לבנות מערכת שתעבד וידאו בזמן אמת עם דיוק גבוה
 1. ****אלגוריתמים מתקדמים**** - הפתרונות הטכניים הכי חדשניים לדיוק מקסימלי
 1. ****בניית MVP**** - איך ליצור פרוטוטיפ ראשוני שעובד ואפשר להדגים אותו
- אני מעמיק במחקר על האתגרים הטכניים הספציפיים של זיהוי אתרוגים בזמן אמת, כולל בעיות תאורה, זוויות צילום, והבחנה מדויקת בין זנים וליקויים!

Your Etrog (citron) real-time identification system research report is ready. It includes advanced computer vision technologies, mobile optimization techniques, deep learning model architectures, and implementation strategies. The report covers technical challenges like varying lighting conditions, detection of blemishes, and differentiating between similar varieties, along with detailed cost-benefit analysis, development timelines, and practical recommendations for an MVP approach.

פיתוח מערכת זיהוי אתרוגים בזמן אמת: מחקר מקיף ויישומי

מערכת זיהוי אתרוגים בזמן אמת לאפליקציות ניידות מציבה אתגרים טכנולוגיים מורכבים הדורשים שילוב של טכנולוגיות Computer Vision מתקדמות, אופטימיזציה למכשירים ניידים, ופתרונות ספציפיים לאתגרי הזיהוי הייחודיים של פירות הדר. המחקר מגלה כי השוק הגלובלי של AI בחקלאות צפוי להגיע ל-4.7 מיליארד דולר עד 2028, עם שיעור צמיחה שנתי של 23.1%. הממצאים מצביעים על כך שמערכות זיהוי פירות מודרניות יכולות להגיע לדיוק של 90-99% עם אופטימיזציה

נכונה למכשירים ניידים.

טכנולוגיות זיהוי בזמן אמת מתקדמות

ארכיטקטורות עיבוד וידאו למכשירים ניידים

****MediaPipe Framework** הוא הפתרון המוביל לעיבוד Computer Vision בזמן אמת:**

- תמיכה רב-פלטפורמית עם תאוצת GPU באמצעות OpenGL ES 3.1 + ב-Android ו-Metal ב-iOS
- ארכיטקטורה מודולרית מבוססת גרף עם "מחשבוני" הניתנים לשימוש חוזר
- ביצועים של 30-60 FPS על מכשירים ניידים מודרניים עם עיבוד בזמן אמת

****OpenCV Mobile Optimizations** מספק:**

- תאוצת חומרה עם תמיכה ב-OpenGL ES
- ניהול זיכרון יעיל עם מגבלות מכשירים ניידים
- אינטגרציה אופטימלית עם Camera API

אופטימיזציה של מודלי Deep Learning

****TensorFlow Lite** מציע טכניקות אופטימיזציה מתקדמות:**

- ****Quantization****: הקטנת גודל המודל פי 4 עם INT8 quantization
- ****FP16 quantization****: הקטנת גודל פי 2 עם אובדן דיוק מינימלי
- ****Dynamic range quantization****: שיפור מהירות הסקה פי 4-1.5 על מעבדי נייד

```
python` ``
```

```
import tensorflow as tf
```

```
Post-training quantization #
```

```
converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
```

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

```
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
```

```
()tflite_model = converter.convert
```

```
`` ``
```

****Knowledge Distillation ו Model Pruning****

- הקטנת פרמטרים של 50-90% עם אובדן דיוק של פחות מ-2%
- שיפור מהירות הסקה משמעותי על מכשירים ניידים

תאוצת חומרה מתקדמת

****GPU Acceleration****:

- ****Metal (iOS)****: שיפור מהירות פי 3-5 לעומת CPU
- ****OpenGL ES (Android)****: שיפור מהירות פי 2-4 עם כיסוי של 52% מהמכשירים

****NPU/DSP Utilization****:

- שיפור מהירות עד פי 25 לעומת CPU
- יעילות אנרגטית פי 5 בהשוואה ל-CPU
- זמינות במכשירי דגל (+A12+, Snapdragon 855)

פתרונות לאתגרים טכניים ספציפיים

התמודדות עם תאורה משתנה

****Color Space Transformation****:

- ****HSV Color Space****: היעיל ביותר לזיהוי פירות מכיוון שהוא מפריד בין רוחב לצבע
- ****L*a*b* Color Space****: ייצוג צבע אחיד מבחינה תפיסתית
- ****YCbCr Color Space****: הפרדה בין עוצמת אור לצבע

****Lighting Normalization****:

- התאמה דינמית של ניגודיות ובהירות
- Histogram equalization סלקטיבי
- שילוב נתוני ספקטרום נראה ו-NIR

זיהוי מזויות וחינים שונים

****Multi-Scale Feature Extraction****:

- ****Feature Pyramid Networks (FPN)****: צבירת תכונות מרמות שונות
- ****Multi-scale attention networks****: התמקדות אוטומטית בתכונות רלוונטיות
- ****Hierarchical contour analysis****: חילוץ מפות קווי מתאר

****Geometric Normalization****:

- ****Affine transformation****: נירמול לתרגום, סיבוב וקנה מידה
- ****Perspective transformation****: תיקון עיוותי זווית צפייה

Scale-invariant features: **תכונות העמידות לשינויי גודל

הבחנה בין זנים דומים

Advanced Feature Extraction:

Texture descriptors: Local Binary Patterns (LBP) **ומסנני Gabor** -

Shape analysis: **תיאורי גיאומטריה כולל יחס גובה-רוחב ועגלגלות

Multi-feature fusion: **שילוב תכונות צבע, מרקם וצורה

Spectral Analysis:

Hyperspectral imaging: **מידע ספקטרלי צר-פס לניתוח הרכב כימי

Multispectral cameras: **חלופה מעשית עם מידע אורך גל סלקטיבי

Near-infrared spectroscopy: **זיהוי איכות פנימית בלתי נראית ב-RGB

זיהוי נקודות שחורות ופגמים

Computer Vision Algorithms:

Region-oriented segmentation: **התמקדות בניגוד בין אזורים פגומים ובריאים

Sobel edge detection: **זיהוי אזורים פגומים באמצעות ניתוח גרדיאנט

Morphological operations: **פעולות פתיחה וסגירה לחידוד גבולות פגמים

Adaptive thresholding: **התאמה דינמית של סף עבור תנאי תאורה שונים

Advanced Detection Methods:

Hyperspectral imaging: **זיהוי פגמים בלתי נראים למצלמות RGB עם דיוק +95%

Deep learning approaches: **מודלי CNN המשיגים precision 93.6% לזיהוי פגמים

Fluorescence imaging: **זיהוי זיהומים בשלב מוקדם באמצעות תכונות פלואורסצנטיות

ארכיטקטורות AI מתקדמות

השוואת ארכיטקטורות זיהוי אובייקטים

YOLO Family Performance:

YOLOv8: **הסקה מהירה (4.1ms עבור YOLOv8n), דיוק בינוני (AP 76.3%)

YOLOv9: **דיוק הגבוה ביותר (mAP@0.50 93.5%), מהירות בינונית (11.5ms)

YOLOv10: **ביצועים מאוזנים (2.4ms עבור YOLOv10n), הפחתת זמן הסקה ב-46%

EfficientNet למכשירים ניידים:

- EfficientNet-B0: 77.1% דיוק ImageNet, 5.3M פרמטרים
- EfficientNet-B4: 82.6% דיוק (לעומת 76.3% עבור ResNet-50)
- EfficientNet-Lite: הסרת squeeze-and-excitation networks, החלפת Swish ב-ReLU6

Vision Transformers אופטימליים

MobileViT Performance:

- MobileViT-S: 78.4% דיוק ImageNet עם 6M פרמטרים
- שיפור של 3.2% לעומת MobileNetV3
- שילוב יעיל של CNN locality עם transformer global modeling

EfficientFormer:

- EfficientFormer-L1: 4.4% דיוק גבוה יותר מ-MobileViT-XS
- מהירות גבוהה פי 2.2 על iPhone NPU, פי 6.8 על iPhone CPU

Multi-Task Learning Architectures

Hybrid Approaches:

- Shared CNN backbone עם multiple task-specific heads
- זיהוי פירות סימולטני (93.24% דיוק) וסיווג (88.66% דיוק)
- הפחתת עומס חישובי באמצעות feature extraction משותף

בניית MVP מעשית

גישת פיתוח מדורגת

2-3 - Phase 1 (Proof of Concept) חודשים:

- זיהוי בסיסי של אתרוג מול לא-אתרוג
- מודל קל (MobileNetV3 או EfficientNet-Lite)
- UI בסיסי עם תצוגת תוצאות ורמת ביטחון
- פונקציונליות offline בסיסית

4-6 - Phase 2 (Core MVP) חודשים:

- הרחבה לזיהוי זנים שונים של אתרוגים

- זיהוי פגמים בסיסי (נקודות שחורות, כתמים)
- הערכת כשרות ראשונית
- שיפור UI/UX עם הנחיות למשתמש

****6-9 Phase 3 (Enhanced MVP) חודשים**:**

- זיהוי מתקדם של פגמים ומאפיינים
- מערכת ציונים מפורטת לכשרות
- מאגר נתונים מקומי מורחב
- מערכת feedback למשתמשים

השוואת פלטפורמות פיתוח

****Flutter + TensorFlow Lite (מומלץ)**:**

- ****יתרונות**:** ביצועים מעולים, codebase יחיד, תמיכה חזקה ב-ML/AI
- ****חסרונות**:** עקומת למידה תלולה יותר, קהילה קטנה יותר
- ****Timeline**:** 3-4 חודשים למ-MVP
- ****ביצועים**:** מעולים למשימות AI כבדות-חישוב

****React Native + ML Kit**:**

- ****יתרונות**:** אקוסיסטם JavaScript, קהילה גדולה, פיתוח מהיר
- ****חסרונות**:** overhead ביצועים, בעיות פוטנציאליות עם ML מורכב
- ****Timeline**:** 2-3 חודשים למ-MVP
- ****ביצועים**:** טובים למרבית המקרים

****Native iOS/Android**:**

- ****יתרונות**:** ביצועים מקסימליים, אופטימיזציות פלטפורמה-ספציפיות
- ****חסרונות**:** דרישת codebase נפרדים, זמן פיתוח ארוך יותר
- ****Timeline**:** 4-6 חודשים למ-MVP

כלים וטכנולוגיות מעשיות

****Camera Integration**:**

- ****Flutter עם תמיכת CameraX עבור Android**:** Camera plugin
- ****opencv-mobile library**:** OpenCV Integration עם JNI bindings

****Model Deployment**:**

- **TensorFlow Lite**: קבצי tflite, dynamic model loading.
- **Model Optimization**: quantization, pruning, compression
- **A/B Testing**: גרסאות מודלים מרובות להשוואה

Performance Monitoring:

- **Firestore Performance Monitoring**: תובנות ביצועים בזמן אמת
- **Custom Metrics**: זמן הסקה, שיעורי דיוק, דיווח קריסות

דרישות נתונים ואימון מודל

כמות נתונים מינימלית

גדלי מאגרי נתונים מומלצים:

- **Small-scale**: 50-200 תמונות לכל קלאס לזיהוי בסיסי
- **Medium-scale**: 500-2,000 תמונות לכל קלאס לביצועים חזקים
- **Large-scale**: 3,000+ תמונות לכל קלאס לדיוק state-of-the-art

דוגמאות ממחקרים:

- **FIDS-30 dataset**: 971 תמונות עם 30 קלאסים (32 תמונות במוצע לקלאס)
- **מאגרי נתונים מותאמים אישית**: 761-3,240 תמונות עבור 8 קטגוריות פירות עם 99.21% דיוק

טכניקות Data Augmentation

Geometric Transformations:

- **Rotation**: זוויות סיבוב אקראיות
- **Flipping**: הפיכות אופקיות ואנכיות
- **Cropping**: חיתוך אקראי וממרכז
- **Scaling**: וריאציות קנה מידה מרובות

Color and Lighting Variations:

- **Brightness adjustment**: רמות תאורה משתנות
- **Contrast modification**: שיפור או הפחתת ניגודיות
- **HSV manipulations**: שינויי גוון ורוויה
- **Normalization**: סטנדרטיזציה של התפלגות צבעים

****Advanced Techniques****

- ****Mixup and CutMix****: שילוב תמונות מרובות
- ****Random erasing****: הסרת patches אקראיים
- ****Gaussian noise****: הוספת רעש מבוקר
- ****Synthetic data generation****: שימוש ב-GANs ליצירת נתונים נוספים

Active Learning Strategies ###

****Uncertainty Sampling****

- ****Adaptive Weighted Uncertainty Sampling (AWUS)****: איזון uncertainty עם random sampling
- ****Entropy-based sampling****: בחירת נקודות נתונים עם entropy גבוה
- ****Bayesian Active Learning****: שימוש ב-Monte Carlo dropout

****Performance Benefits****

- הפחתת עומס annotation של למעלה מ-85%
- מחזורי פיתוח מהירים יותר
- הכללה משופרת על מקרים נדירים

מדידת דיוק וביצועים

מטריקות סיווג

****Core Metrics****

- ****Accuracy****: נכונות כוללת (90-99.5% טיפוס לזיהוי פירות)
- ****Precision****: $TP/(TP+FP)$ - דיוק של חיזויים חיוביים
- ****Recall****: $TP/(TP+FN)$ - יכולת זיהוי כל המקרים החיוביים
- ****F1-Score****: ממוצע הרמוני של precision ו-recall

****Advanced Metrics****

- ****Mean Average Precision (mAP)****: שטח מתחת לעקומת precision-recall
- ****Intersection over Union (IoU)****: חפיפה בין חיזוי ו-ground truth
- ****Confusion Matrix****: פירוט מפורט של שגיאות סיווג

Real-Time Performance Metrics ###

****Speed and Efficiency****

- ****FPS****: מהירות עיבוד לאפליקציות זמן אמת
- ****Latency****: זמן עיכוב בין קלט לפלט
- ****Memory Usage****: דרישות RAM ואחסון
- ****Inference Time****: זמן הנדרש לחיזוי יחיד

****Mobile-Specific Benchmarks****

- ****Target Performance****: <500ms זמן הסקה לזיהוי בזמן אמת
- ****Memory Constraints****: <150MB שימוש בזיכרון שיא
- ****Battery Impact****: <2% בזבוז סוללה לשעת שימוש

A/B Testing Methodologies

****Model Comparison****

- השוואת גרסאות מודלים שונות
- מדידת accuracy vs. speed tradeoffs
- ניתוח ביצועים על מכשירים שונים
- בדיקת robustness בתנאים שונים

****Evaluation Protocols****

- ****Cross-validation****: בדרך כלל k=5 או k=10 folds
- ****Stratified splits****: שמירה על התפלגות קלאסים
- ****Holdout validation****: סט בדיקה עצמאי
- ****Statistical significance****: ריצות מרובות עם random seeds שונים

שיקולים עלות-תועלת

עלויות פיתוח מפורטות

****צוות הפיתוח הליבה****

- ****AI/ML Engineer****: \$120-200k שנתי (ארה"ב), \$30-70/שעה (מיקור חוץ)
- ****Mobile Developer****: \$80-150k שנתי, \$25-60/שעה (מיקור חוץ)
- ****Computer Vision Specialist****: \$100-180k שנתי, \$40-80/שעה
- ****Data Scientist****: \$90-160k שנתי, \$35-70/שעה

****עלויות פיתוח לפי מורכבות****

****אפליקציית זיהוי פירות פשוטה**:**

- ****פיתוח****: \$50,000-150,000

- ****Timeline****: 3-6 חודשים

- ****תכונות****: זיהוי תמונות בסיסי, מאגר מינים מוגבל

****אפליקציית AI חקלאית מתקדמת**:**

- ****פיתוח****: \$200,000-500,000

- ****Timeline****: 8-18 חודשים

- ****תכונות****: זיהוי מינים מרובים, זיהוי מחלות, חיזוי יכול

ROI Analysis ###

****אסטרטגיות מונטיזציה**:**

- ****Freemium Model****: זיהוי בסיסי חינם, תכונות מתקדמות \$2.99-9.99/חודש

- ****Subscription Tiers****: \$4.99 (בסיסי), \$19.99/חודש (מקצועי)

- ****Pay-per-use****: \$0.10-0.50 לזיהוי עבור משתמשים מקצועיים

****חזויי הכנסות**:**

- ****שנה 1****: \$50,000-200,000 (1,000-5,000 משתמשים משלמים)

- ****שנה 3****: \$500,000-2,000,000 (10,000-50,000 משתמשים משלמים)

- ****שנה 5****: \$2,000,000-10,000,000 (50,000-200,000 משתמשים משלמים)

גורמי קיימות ארוכת טווח

****Value Proposition דרישות**:**

- ****דיוק****: מינימום 90% לביצועים מסחריים

- ****מהירות****: זמן תגובה של פחות מ-3 שניות

- ****Offline Capability****: חיוני לאזורים כפריים/מרוחקים

- ****תמיכה רב-לשונית****: קריטית לחדירה לשוק גלובלי

Case Studies רלוונטיים

פרויקטים מוצלחים

****מימושים אקדמיים**:**

- **Apple Black Rot Detection**: Deep CNN עם 90.4% דיוק בסיווג חומרת מחלה
- **Tomato Disease Detection**: אלגוריתם YOLOv3 משופר עם 92.3% שיעור זיהוי
- **Mango Yield Prediction**: מערכת computer vision עם 11,000+ תמונות מתווגות

****מימושים מסחריים**:**

- **PlantNet**: הסטנדרט הזהב לזיהוי צמחים עם 45+ פלורות ו-46,050 מינים
- **PictureThis**: מובילה מסחרית עם 73% דיוק בהערכות 2023
- **Blue River Technology**: טכנולוגיית "See & Spray" עם הפחתת שימוש בקוטלי עשבים של עד 90%

לקחים מפרויקטים דומים

****גורמי הצלחה**:**

- **איכות נתונים**: השקעה כבדה במאגרי נתונים מגוונים ואיכותיים
- **בדיקות שטח**: ולידציה נרחבת בתנאים אמיתיים לפני השקה
- **למידה מתמשכת**: מודלים המשתפרים עם השימוש
- **שיתוף עם מומחי תחום**: שילוב של מומחיות חקלאית מהיום הראשון

****מלכודות נפוצות**:**

- **בעיות איכות נתונים**: איכות תמונה גרועה המובילה לתוצאות לא מדויקות
- **Overfitting**: מודלים הפועלים היטב במעבדה אך נכשלים בתנאי שטח
- **מגבלות חישוביות**: הערכת יתר של יכולות מכשירים ניידים
- **הנחות קישוריות**: הנחה על אינטרנט אמין באזורים כפריים

המלצות מעשיות ליישום

אסטרטגיית פיתוח מומלצת

****שלב 1: MVP בסיסי (3-4 חודשים)****

1. **בחירת טכנולוגיה**: Flutter + TensorFlow Lite
1. **מודל ראשוני**: MobileNetV3-Small עם transfer learning
1. **תכונות ליבה**: זיהוי אתרוג בסיסי, UI פשוט
1. **מאגר נתונים**: 500-1,000 תמונות אתרוגים מתווגות

****שלב 2: שיפור דיוק (2-3 חודשים נוספים)****

1. ****הרחבת מאגר נתונים****: 2,000-3,000 תמונות
1. ****זיהוי פגמים****: זיהוי נקודות שחורות וכתמים
1. ****אופטימיזציה****: quantization ו-pruning למהירות
1. ****בדיקות שטח****: טסטים עם משתמשים אמיתיים

****שלב 3: תכונות מתקדמות (3-4 חודשים נוספים)****

1. ****Multi-task learning****: זיהוי סימולטני של זן ופגמים
1. ****הערכת כשרות****: מערכת ציונים מפורטת
1. ****אופטימיזציה למכשירים****: תמיכה בחומרת NPU/GPU
1. ****מערכת feedback****: איסוף נתונים מתמשיך

ספציפיקציות טכניות מומלצות

****ארכיטקטורת מודל****:

```
python``
# מודל אופטימלי לזיהוי אתרוגים
def create_citron_model():
    base_model = tf.keras.applications.MobileNetV3Small
        ,input_shape=(224, 224, 3)
        ,include_top=False
        'weights='imagenet'
    (

    ])model = tf.keras.Sequential
        ,base_model
        ,()tf.keras.layers.GlobalAveragePooling2D
        ,tf.keras.layers.Dense(128, activation='relu')
        ,tf.keras.layers.Dropout(0.2)
    tf.keras.layers.Dense(num_citron_classes, activation='softmax')
    ([

    return model
    ``
```

****מטרות ביצועים****:

- ****דיוק****: +95% לזיהוי אתרוג vs. לא אתרוג
- ****מהירות****: >300ms זמן הסקה
- ****גודל מודל****: >10MB לאחר quantization

- ****שימוש בזיכרון****: >100MB runtime

****אסטרטגיית נתונים****:

- ****מינימום****: 1,000 תמונות אתרוגים איכותיות

- ****מגוון****: תנאי תאורה, זוויות, רקעים שונים

- ****x10 Augmentation****: הגדלת מאגר באמצעות טרנספורמציות

- ****Active Learning****: בחירה חכמה של תמונות לתיוג

הסיכום והמסקנות

המחקר מצביע על כך שפיתוח מערכת זיהוי אתרוגים מוצלחת דורש שילוב של מומחיות טכנולוגית, הבנה עמיקה של הדרישות הספציפיות, גישה מדורגת המתחילה ב-MVP פשוט ומתפתח לפתרון מתקדם. עם השקעה נכונה בנתונים, אופטימיזציה טכנולוגית מתאימה, וביצוע מדויק של התהליך המדורג, ניתן להשיג מערכת עם דיוק של 95%+ המתאימה לשימוש מעשי בזיהוי אתרוגים.

****גורמי הצלחה קריטיים****:

- דיוק טכני של 90%+ עם אופטימיזציה למכשירים ניידים

- יכולת עבודה offline לאזורים כפריים

- שילוב מומחיות תחום מהיום הראשון

- מודלים עסקיים ניתנים להרחבה

- פיתוח מונע קהילה עם ולידציה מתמשכת

השילוב של שיפור חומרת מובייל, יכולות Edge AI, וגידול באימון טכנולוגיות חק לאיות יוצר סביבה נוחה למתחדשים חדשים שיכולים לבצע היטב גם במימד הטכני וגם במימד העסקי.

```
pubspec.yaml //
```

```
*/
```

```
name: etrog_identifier
```

```
description: AI-powered Etrog identification app
```

```
version: 1.0.0+1
```

```
:environment
```

```
'sdk: ' >=3.0.0 <4.0.0
```

```
"flutter: " >=3.10.0
```

```
:dependencies
```

```
:flutter
```

```
  sdk: flutter
```

```

cupertino_icons: ^1.0.6
  camera: ^0.10.5+5
  tflite_flutter: ^0.10.4
    image: ^4.1.3
  path_provider: ^2.1.1
  permission_handler: ^11.0.1
    provider: ^6.1.1
  :flutter_localizations
    sdk: flutter
    intl: ^0.18.1

:dev_dependencies
  :flutter_test
    sdk: flutter
  flutter_lints: ^3.0.1

:flutter
uses-material-design: true
  :assets
    /assets/models -
    /assets/images -
    /assets/lang -
    :fonts
    family: Heebo -
    :fonts
  asset: fonts/Heebo-Regular.ttf -
  asset: fonts/Heebo-Bold.ttf -
  weight: 700
/*

lib/main.dart //
;import 'package:flutter/material.dart
;import 'package:flutter/services.dart
;import 'package:provider/provider.dart
;import 'package:flutter_localizations/flutter_localizations.dart

;import 'core/app_config.dart
;import 'core/localization/app_localizations.dart
;import 'core/theme/app_theme.dart
;import 'features/splash/splash_screen.dart
;import 'features/home/home_screen.dart

```

```

;import 'features/camera/camera_screen.dart
;import 'features/results/results_screen.dart
;import 'providers/app_state_provider.dart
;import 'providers/camera_provider.dart
;import 'providers/ml_provider.dart

} void main() async
;()WidgetsFlutterBinding.ensureInitialized

Lock orientation to portrait //
])await SystemChrome.setPreferredOrientations
,DeviceOrientation.portraitUp
,DeviceOrientation.portraitDown
;([

Initialize app configuration //
;()await AppConfig.initialize

;runApp(const EtrogIdentifierApp())
{

} class EtrogIdentifierApp extends StatelessWidget
;const EtrogIdentifierApp({super.key})

override@
} Widget build(BuildContext context)
)return MultiProvider
]:providers
,ChangeNotifierProvider(create: (*) => AppStateProvider())
,ChangeNotifierProvider(create: (*) => CameraProvider())
,ChangeNotifierProvider(create: (_) => MLProvider())
,[
)<child: Consumer<AppStateProvider
} builder: (context, appState, child)
)return MaterialApp
title: 'זיהוי אתרוגים',
,debugShowCheckedModeBanner: false

...

Localization //
] localizationsDelegates: const

```

```

        ,AppLocalizations.delegate
        ,GlobalMaterialLocalizations.delegate
        ,GlobalWidgetsLocalizations.delegate
        ,GlobalCupertinoLocalizations.delegate
    ],
    ] supportedLocales: const
    Locale('he', 'IL'), // Hebrew
    Locale('en', 'US'), // English
    ],
    ,locale: appState.currentLocale

    Theme //
    ,theme: AppTheme.lightTheme
    ,darkTheme: AppTheme.darkTheme
    ,themeMode: appState.themeMode

    Routes //
    , '/' :initialRoute
    } :routes

    ,()const SplashScreen <= (context) : '/'
    ,()home': (context) => const HomeScreen/'
    ,()camera': (context) => const CameraScreen/'
    ,()results': (context) => const ResultsScreen/'
    ,{
    ;(
    ,{
    ,(
    ;(
    ...

    {
    {

lib/core/app_config.dart //
; 'import 'dart:io
; 'import 'package:flutter/services.dart
; 'import 'package:path_provider/path_provider.dart

    } class AppConfig
; 'static const String modelFileName = 'etrog_classifier_v1.tflite
; 'static const String labelsFileName = 'etrog_labels.txt

```

```

        ;static const double modelInputSize = 224.0
        ;static const int modelBatchSize = 1
        ;static const int modelChannels = 3

        Performance settings //
        ;static const int cameraFPS = 30
        ;static const double confidenceThreshold = 0.7
        static const int maxInferenceTime = 500; // milliseconds

        App settings //
        ;'static const String appVersion = '1.0.0
        ;static const bool debugMode = true

        ;static late String modelPath
        ;static late String labelsPath
        ;static late List<String> labels

    } static Future<void> initialize() async
    } try
        Copy model from assets to local storage //
        ;()final appDir = await getApplicationDocumentsDirectory
        ;'modelPath = '${appDir.path}/${modelFileName}
        ;'labelsPath = '${appDir.path}/${labelsFileName}

        ...

        Copy model file //
        ;final modelData = await rootBundle.load('assets/models/$modelFileName')
        ;final modelFile = File(modelPath)
        ;await modelFile.writeAsBytes(modelData.buffer.asUint8List())

        Copy and load labels //
        ;final labelsData = await rootBundle.loadString('assets/models/$labelsFileName')
        ;final labelsFile = File(labelsPath)
        ;await labelsFile.writeAsString(labelsData)

        labels = labelsData.split('\n')
        where((line) => line.trim().isEmpty).
        ;()toList.

        ;print('AppConfig initialized successfully')
        ;print('Model path: $modelPath')

```

```

;print('Labels count: ${labels.length}')

    } catch (e) {
;print('Error initializing AppConfig: $e')
;rethrow
    }
    ...

    {
    {

lib/core/theme/app_theme.dart //
;import 'package:flutter/material.dart

    } class AppTheme
        Color palette //
        static const Color primaryColor = Color(0xFF2E7D32); // Green
static const Color secondaryColor = Color(0xFFFF8F00); // Orange
;static const Color backgroundColor = Color(0xFFF5F5F5)
;static const Color surfaceColor = Colors.white
;static const Color errorColor = Color(0xFFD32F2F)
;static const Color successColor = Color(0xFF388E3C)
;static const Color warningColor = Color(0xFFF57C00)

        Text colors //
;static const Color textPrimary = Color(0xFF212121)
;static const Color textSecondary = Color(0xFF757575)
;static const Color textOnPrimary = Colors.white

    } static ThemeData get lightTheme
        )return ThemeData
        ,useMaterial3: true
    )colorScheme: ColorScheme.fromSeed
        ,seedColor: primaryColor
        ,brightness: Brightness.light
        ,primary: primaryColor
        ,secondary: secondaryColor
        ,surface: surfaceColor
        ,background: backgroundColor
        ,error: errorColor
        ,(

```

...

```

        AppBar Theme //
    )AppBarTheme: const AppBarTheme
        ,elevation: 0
        ,centerTitle: true
        ,backgroundColor: primaryColor
        ,foregroundColor: textOnPrimary
        )titleTextStyle: TextStyle
            ,fontSize: 20
            ,fontWeight: FontWeight.bold
            ,color: textOnPrimary
            ,fontFamily: 'Heebo
        ,(
        ,(

        Card Theme //
    )cardTheme: CardTheme
        ,elevation: 4
        )shape: RoundedRectangleBorder
        ,borderRadius: BorderRadius.circular(12)
        ,(
        ,margin: const EdgeInsets.all(8)
        ,(

        Elevated Button Theme //
    )elevatedButtonTheme: ElevatedButtonThemeData
        )style: ElevatedButton.styleFrom
            ,elevation: 2
        ,padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12)
        )shape: RoundedRectangleBorder
        ,borderRadius: BorderRadius.circular(8)
        ,(
        )textStyle: const TextStyle
            ,fontSize: 16
            ,fontWeight: FontWeight.w600
            ,fontFamily: 'Heebo
        ,(
        ,(
        ,(
```

```

        Text Theme //
    )textTheme: const TextStyle
        )headlineLarge: TextStyle
            ,fontSize: 32
        ,fontWeight: FontWeight.bold
            ,color: textPrimary
            ,fontFamily: 'Heebo
                ,(
        )headlineMedium: TextStyle
            ,fontSize: 24
        ,fontWeight: FontWeight.w600
            ,color: textPrimary
            ,fontFamily: 'Heebo
                ,(
        )titleLarge: TextStyle
            ,fontSize: 20
        ,fontWeight: FontWeight.w500
            ,color: textPrimary
            ,fontFamily: 'Heebo
                ,(
        )bodyLarge: TextStyle
            ,fontSize: 16
            ,color: textPrimary
            ,fontFamily: 'Heebo
                ,(
        )bodyMedium: TextStyle
            ,fontSize: 14
            ,color: textSecondary
            ,fontFamily: 'Heebo
                ,(
                ,(

    Input Decoration Theme //
)inputDecorationTheme: InputDecoration
    )border: OutlineInputBorder
        ,borderRadius: BorderRadius.circular(8)
            ,(
        )focusedBorder: OutlineInputBorder
            ,borderRadius: BorderRadius.circular(8)
        ,borderSide: const BorderSide(color: primaryColor, width: 2)
            ,(

```

```

        , (
        ; (
        ...

    {

        } static ThemeData get darkTheme
            )return ThemeData
            ,useMaterial3: true
        )colorScheme: ColorScheme.fromSeed
            ,seedColor: primaryColor
            ,brightness: Brightness.dark
            , (
            Add dark theme customizations here //
            ; (
            {
            {

lib/providers/app_state_provider.dart //
;import 'package:flutter/material.dart

} class AppStateProvider extends ChangeNotifier
    ;Locale _currentLocale = const Locale('he', 'IL')
;ThemeMode _themeMode = ThemeMode.system
    ;bool _isInitialized = false
    ;String? _lastError

        Getters //
    ;Locale get currentLocale => _currentLocale
;ThemeMode get themeMode => _themeMode
    ;bool get isInitialized => _isInitialized
    ;String? get lastError => _lastError

        Methods //
    } void setLocale(Locale locale)
    } if (_currentLocale != locale)
        ;currentLocale = locale_
        ;()notifyListeners
        {
        {

```

```

    } void setThemeMode(ThemeMode mode)
      } if (_themeMode != mode)
        ;themeMode = mode_
        ;()notifyListeners
        {
        {

```

```

    } void setInitialized(bool initialized)
      } if (!_isInitialized != initialized)
        ;isInitialized = initialized_
        ;()notifyListeners
        {
        {

```

```

    } void setError(String? error)
      ;lastError = error_
      ;()notifyListeners
      {

```

```

    } ()void clearError
      ;lastError = null_
      ;()notifyListeners
      {
      {

```

```

lib/providers/ml_provider.dart //
    ;'import 'dart:io
    ;'import 'dart:typed_data
    ;'import 'package:flutter/material.dart
; 'import 'package:tflite_flutter/tflite_flutter.dart
    ;import 'package:image/image.dart' as img

    ;'import '../core/app_config.dart
    ;'import '../models/etrog_result.dart
    ;'import '../services/image_processor.dart

```

```

} class MLProvider extends ChangeNotifier
    ;Interpreter? _interpreter
    ;bool _isModelLoaded = false
    ;bool _isProcessing = false
    ;String? _lastError

```

```

Getters //
;bool get isModelLoaded => _isModelLoaded
;bool get isProcessing => _isProcessing
;String? get lastError => _lastError

Initialize TensorFlow Lite model //
} Future<void> initializeModel() async
    } try
        ;setProcessing(true)_
        ;()clearError_
        ...

Load the model //
;interpreter = await Interpreter.fromFile(File(AppConfig.modelPath))_

Allocate tensors //
;()interpreter!.allocateTensors_

Verify model input/output shapes //
;final inputShape = _interpreter!.getInputTensor(0).shape
;final outputShape = _interpreter!.getOutputTensor(0).shape

;print('Model loaded successfully')
;print('Input shape: $inputShape')
;print('Output shape: $outputShape')

;isModelLoaded = true_

    } catch (e) {
;setError('Failed to load model: $e')_
;print('Error loading model: $e')
    } finally {
;setProcessing(false)_
    {
        ...
    }

Classify etrog from image //
} Future<EtrogResult?> classifyImage(String imagePath) async

```

```

        } if (!_isModelLoaded || _interpreter == null)
            ;setError('Model not loaded')_
            ;return null
        {
            ...
        } try
            ;setProcessing(true)_
            ;()clearError_

        ;()final stopwatch = Stopwatch()××start

            Process image //
;final processedImage = await ImageProcessor.preprocessImage(imagePath)
            } if (processedImage == null)
                ;setError('Failed to process image')_
                ;return null
            {

                Prepare input tensor //
                ;final input = _prepareInputTensor(processedImage)
                final output = List×filled(AppConfig×labels×length, 0.0)
                ;reshape([1, AppConfig.labels.length]).

                Run inference //
                ;interpreter!.run(input, output)_

                ;()stopwatch.stop

                Process output //
;final result = _processOutput(output[0], stopwatch.elapsedMilliseconds)

                ;print('Inference completed in ${stopwatch.elapsedMilliseconds}ms')
                ;return result

            } catch (e) {
                ;setError('Classification failed: $e')_
                ;print('Error during classification: $e')
                ;return null
            } finally {
                ;setProcessing(false)_

```

```

        {
        ...

        {

        Prepare input tensor from processed image //
    } List<List<List<List<double>>>> *prepareInputTensor(img.Image image)
        )final input = List.generate
            ,1
            )List.generate <= (*)
            ,()AppConfig.modelInputSize.toInt
            )List.generate <= (*)
            ,()AppConfig.modelInputSize.toInt
            ,List.filled(3, 0.0) <= (*)
            ,(
            ,(
            ;(
            ...

            } for (int y = 0; y < image.height; y++)
            } for (int x = 0; x < image.width; x++)
                ;final pixel = image.getPixel(x, y)
                input[0][y][x][0] = pixel.r / 255.0; // Red
                input[0][y][x][1] = pixel.g / 255.0; // Green
                input[0][y][x][2] = pixel.b / 255.0; // Blue
                {
                {

                ;return input
                ...

                {

                Process model output to EtrogResult //
    } EtrogResult _processOutput(List<double> output, int inferenceTime)
        Find the class with highest confidence //
            ;double maxConfidence = 0.0
            ;int maxIndex = 0
            ...

            } for (int i = 0; i < output.length; i++)

```

```

        } if (output[i] > maxConfidence)
            ;maxConfidence = output[i]
            ;maxIndex = i
            {
            {

                Create confidence map //
                ;{}<final confidenceMap = <String, double
            } for (int i = 0; i < AppConfig.labels.length && i < output.length; i++)
                ;confidenceMap[AppConfig.labels[i]] = output[i]
                {

                    Determine result status //
                    ;EtrogStatus status
                } if (maxConfidence < AppConfig.confidenceThreshold)
                    ;status = EtrogStatus.uncertain
                    } else {
                ;()final label = AppConfig.labels[maxIndex].toLowerCase
                } (label.contains('kosher') || ('כשר')label.contains) if
                    ;status = EtrogStatus.kosher
                } (label.contains('invalid') || ('פסול')label.contains) else if {
                    ;status = EtrogStatus.invalid
                    } else {
                ;status = EtrogStatus.uncertain
                {
                {

                    )return EtrogResult
                ,predictedClass: AppConfig.labels[maxIndex]
                ,confidence: maxConfidence
                ,status: status
                ,confidenceMap: confidenceMap
                ,inferenceTime: inferenceTime
                ,()timestamp: DateTime.now
                ;(
                ...

                {

                    Helper methods //
                } void _setProcessing(bool processing)

```

```

    } if (_isProcessing != processing)
        ;isProcessing = processing_
        ;()notifyListeners
        {
        {

    } void _setError(String error)
        ;lastError = error_
        ;()notifyListeners
        {

        } ()void _clearError
    } if (_lastError != null)
        ;lastError = null_
        ;()notifyListeners
        {
        {

        override@
        } ()void dispose
        ;()interpreter?.close_
        ;()super.dispose
        {
        {

lib/models/etrog_result.dart //
    } enum EtrogStatus
        ,kosher
        ,invalid
        ,uncertain
        {

        } class EtrogResult
        ;final String predictedClass
        ;final double confidence
        ;final EtrogStatus status
;final Map<String, double> confidenceMap
        ;final int inferenceTime
        ;final DateTime timestamp
        ;final String? variety
;final List<String>? detectedDefects

```

```
;final Map<String, dynamic>? additionalInfo
```

```
    })const EtrogResult  
    ,required this.predictedClass  
    ,required this.confidence  
    ,required this.status  
    ,required this.confidenceMap  
    ,required this.inferenceTime  
    ,required this.timestamp  
    ,this.variety  
    ,this.detectedDefects  
    ,this.additionalInfo  
    ;{
```

```
    Helper getters //
```

```
    ;bool get isKosher => status == EtrogStatus.kosher  
    ;bool get isInvalid => status == EtrogStatus.invalid  
    ;bool get isUncertain => status == EtrogStatus.uncertain
```

```
    } String get statusText  
    } switch (status)  
    :case EtrogStatus.kosher  
        ;'כשר' return  
    :case EtrogStatus.invalid  
        ;'פסול' return  
    :case EtrogStatus.uncertain  
        ;'לא ברור' return  
    {  
    {
```

```
    } Color get statusColor  
    } switch (status)  
    :case EtrogStatus.kosher  
        return const Color(0xFF4CAF50); // Green  
    :case EtrogStatus.invalid  
        return const Color(0xFFFF44336); // Red  
    :case EtrogStatus.uncertain  
        return const Color(0xFFFF9800); // Orange  
    {  
    {
```

```
;'%String get confidencePercentage => '${(confidence * 100).toStringAsFixed(1)}
```

```
        Convert to JSON for storage //
    } ()Map<String, dynamic> toJson
        } return
    ,predictedClass': predictedClass'
    ,confidence': confidence'
    ,()status': status.toString'
    ,confidenceMap': confidenceMap'
    ,inferenceTime': inferenceTime'
    ,()timestamp': timestamp.toIso8601String'
    ,variety': variety'
    ,detectedDefects': detectedDefects'
    ,additionalInfo': additionalInfo'
    ;{
    {

        Create from JSON //
    } factory EtrogResult.fromJson(Map<String, dynamic> json)
        )return EtrogResult
    ,predictedClass: json['predictedClass']
    ,confidence: json['confidence']
    )status: EtrogStatus.values.firstWhere
    ,extoString() == json['status'] <= (e)
    ,(
    ,confidenceMap: Map<String, double>.from(json['confidenceMap'])
    ,inferenceTime: json['inferenceTime']
    ,timestamp: DateTime.parse(json['timestamp'])
    ,variety: json['variety']
    ,()<detectedDefects: json['detectedDefects']?.cast<String>
    ,additionalInfo: json['additionalInfo']
    ;(
    {
    {
```

```
lib/services/image_processor.dart //
    ;'import 'dart:io
;import 'package:image/image.dart' as img
    ;'import '../core/app_config.dart
```

```
    } class ImageProcessor
```

```

        Preprocess image for model input //
    } static Future<img.Image?> preprocessImage(String imagePath) async
        } try
            Read image file //
            ;final imageFile = File(imagePath)
            ;()final imageBytes = await imageFile.readAsBytes

            ...

            Decode image //
;img.Image? image = imgxdecodeImage(imageBytes)
            ;if (image == null) return null

            Apply preprocessing pipeline //
            ;image = _resizeImage(image)
;image = _normalizeImage(image)
            ;image = _enhanceImage(image)

            ;return image
            } catch (e) {
;print('Error preprocessing image: $e')
            ;return null
            {
                ...
            }

            {

            Resize image to model input size //
    } static img.Image _resizeImage(img.Image image)
        ;()final size = AppConfigxmodelInputSizetoInt
        )return img.copyResize
            ,image
            ,width: size
            ,height: size
        ,interpolation: img.Interpolation.cubic
        ;(
        {

            Normalize image values //
    } static img.Image _normalizeImage(img.Image image)
        Apply histogram equalization for better contrast //
        )return img.adjustColor

```

```

        ,image
        ,contrast: 1.1
        ,brightness: 1.05
        ;(
        {

        Enhance image quality //
    } static img.Image _enhanceImage(img.Image image)
        Apply slight sharpening //
        )return img.convolution
            ,image
            ] :filter
            ,0 ,1- ,0
            ,1- ,5 ,1-
            ,0 ,1- ,0
            ,[
            ,div: 1
            ;(
            {

        Extract color features for additional analysis //
    } static Map<String, double> extractColorFeatures(img.Image image)
        ;int totalPixels = image.width * image.height
        ;double totalR = 0, totalG = 0, totalB = 0
        ;double totalH = 0, totalS = 0, totalV = 0

        ...

        } for (int y = 0; y < image.height; y++)
        } for (int x = 0; x < image.width; x++)
            ;final pixel = image.getPixel(x, y)

            RGB values //
            ;totalR += pixel.r
            ;totalG += pixel.g
            ;totalB += pixel.b

            Convert to HSV //
        ;final hsv = _rgbToHsv(pixel.r, pixel.g, pixel.b)
            ;!totalH += hsv['h']
            ;!totalS += hsv['s']
            ;!totalV += hsv['v']

```

```

        {
        {

        } return
        ,avgR': totalR / totalPixels / 255.0'
        ,avgG': totalG / totalPixels / 255.0'
        ,avgB': totalB / totalPixels / 255.0'
        ,avgH': totalH / totalPixels'
        ,avgS': totalS / totalPixels'
        ,avgV': totalV / totalPixels'
        ;{
        ...

        {

        Convert RGB to HSV //
    } static Map<String, double> _rgbToHsv(num r, num g, num b)
        ;r = r / 255.0
        ;g = g / 255.0
        ;b = b / 255.0

        ...

        ;final max = [r, g, b].reduce((a, b) => a > b ? a : b)
        ;final min = [r, g, b].reduce((a, b) => a < b ? a : b)
        ;final diff = max - min

        ;double h = 0
        } if (diff != 0)
        } if (max == r)
        ;h = (60 * ((g - b) / diff) + 360) % 360
        } else if (max == g) {
        ;h = (60 * ((b - r) / diff) + 120) % 360
        } else if (max == b) {
        ;h = (60 * ((r - g) / diff) + 240) % 360
        {
        {

        ;final s = max == 0 ? 0 : diff / max
        ;final v = max

        ;return {'h': h, 's': s, 'v': v}

```

...

{
{

```
pubspec.yaml //  
*/  
name: etrog_identifier  
description: AI-powered Etrog identification app  
version: 1.0.0+1  
  
environment  
  sdk: '>=3.0.0 <4.0.0'  
  flutter: ">=3.10.0"  
  
dependencies  
  flutter  
    sdk: flutter  
  cupertino_icons: ^1.0.6  
  camera: ^0.10.5+5  
  tflite_flutter: ^0.10.4  
  image: ^4.1.3  
  path_provider: ^2.1.1  
  permission_handler: ^11.0.1  
  provider: ^6.1.1  
  flutter_localizations  
    sdk: flutter  
    intl: ^0.18.1  
  
dev_dependencies  
  flutter_test  
    sdk: flutter  
  flutter_lints: ^3.0.1  
  
flutter  
uses-material-design: true  
assets  
  /assets/models -  
  /assets/images -
```

```

        /assets/lang -
        :fonts
        family: Heebo -
        :fonts
        asset: fonts/Heebo-Regular.ttf -
        asset: fonts/Heebo-Bold.ttf -
        weight: 700
        /*

lib/main.dart //
;import 'package:flutter/material.dart
;import 'package:flutter/services.dart
;import 'package:provider/provider.dart
;import 'package:flutter_localizations/flutter_localizations.dart

;import 'core/app_config.dart
;import 'core/localization/app_localizations.dart
;import 'core/theme/app_theme.dart
;import 'features/splash/splash_screen.dart
;import 'features/home/home_screen.dart
;import 'features/camera/camera_screen.dart
;import 'features/results/results_screen.dart
;import 'providers/app_state_provider.dart
;import 'providers/camera_provider.dart
;import 'providers/ml_provider.dart

} void main() async
;()WidgetsFlutterBinding.ensureInitialized

Lock orientation to portrait //
])await SystemChrome.setPreferredOrientations
,DeviceOrientation.portraitUp
,DeviceOrientation.portraitDown
;([

Initialize app configuration //
;()await AppConfig.initialize

;runApp(const EtrogIdentifierApp())
{

```

```

    } class EtrogIdentifierApp extends StatelessWidget
      ;const EtrogIdentifierApp({super.key})

      override@
    } Widget build(BuildContext context)
      )return MultiProvider
        ] :providers
      ,ChangeNotifierProvider(create: (*) => AppStateProvider())
      ,ChangeNotifierProvider(create: (*) => CameraProvider())
      ,ChangeNotifierProvider(create: (_) => MLProvider())
      ,[
    )<child: Consumer<AppStateProvider
      } builder: (context, appState, child)
    )return MaterialApp
      title: 'זיהוי אתרוגים',
      debugShowCheckedModeBanner: false

      ...

      Localization //
    ] localizationsDelegates: const
      ,AppLocalizations.delegate
      ,GlobalMaterialLocalizations.delegate
      ,GlobalWidgetsLocalizations.delegate
      ,GlobalCupertinoLocalizations.delegate
      ,[
    ] supportedLocales: const
      Locale('he', 'IL'), // Hebrew
      Locale('en', 'US'), // English
      ,[
    ,locale: appState.currentLocale

      Theme //
    ,theme: AppTheme.lightTheme
    ,darkTheme: AppTheme.darkTheme
    ,themeMode: appState.themeMode

      Routes //
    ,'/':initialRoute
    } :routes
    ,()const SplashScreen <= (context) : '/'
    ,()home': (context) => const HomeScreen/'

```

```

,()camera': (context) => const CameraScreen/'
,()results': (context) => const ResultsScreen/'
    ,{
    ;(
    ,{
    ,(
    ;(
    ...

    {
    {

lib/features/splash/splash_screen.dart //
;import 'package:flutter/material.dart'
;import 'package:provider/provider.dart'
;import '../providers/app_state_provider.dart'
;import '../providers/ml_provider.dart'
;import '../core/app_config.dart'

} class SplashScreen extends StatefulWidget
;const SplashScreen({super.key})

    override@
;()State<SplashScreen> createState() => _SplashScreenState
{

<class _SplashScreenState extends State<SplashScreen
    } with SingleTickerProviderStateMixin
    ;late AnimationController _controller
    ;late Animation<double> _fadeAnimation
    ;late Animation<double> _scaleAnimation

    override@
    } ()void initState
    ;()super.initState

    ...

    )controller = AnimationController_
,duration: const Duration(seconds: 2)
,vsync: this
;(

```

```

        )<fadeAnimation = Tween<double_
            ,begin: 0.0
            ,end: 1.0
        )animate(CurvedAnimation.(
            ,parent: _controller
            ,curve: Curves.easeIn
        );(

    )<scaleAnimation = Tween<double_
        ,begin: 0.5
        ,end: 1.0
    )animate(CurvedAnimation.(
        ,parent: _controller
        ,curve: Curves.elasticOut
    );(

        ;()initializeApp_
        ...

    {

    } Future<void> _initializeApp() async
;()<final appState = context.read<AppStateProvider
;()<final mlProvider = context.read<MLProvider

    ...

    } try
;()controller.forward_

        Initialize ML model //
;()await mlProvider.initializeModel

    } if (!mlProvider.isModelLoaded)
;throw Exception('Failed to load ML model')
    {

;appState.setInitialized(true)

    Wait for animation to complete //
;()await _controller.forward

```

```

;await Future.delayed(const Duration(milliseconds: 500))

      } if (mounted)
;Navigator.of(context).pushReplacementNamed('/home')
      {

      } catch (e) {
;appState.setError('Initialization failed: $e')
;showErrorDialog(e.toString())_
      {
      ...

      {

    } void _showErrorDialog(String error)
      )showDialog
      ,context: context
      ,barrierDismissible: false
    )builder: (context) => AlertDialog
      ,('שגיאה')title: const Text
,('n$error\:\:האפליקציה באתחול האפליקציה')content: Text
      ] :actions
      )TextButton
      } () :onPressed
;()Navigator.of(context).pop
      initializeApp(); // Retry_
      ,{
      ,('נסה שוב')child: const Text
      ,(
      ,[
      ,(
      ;(
      {

      override@
      } ()void dispose
;()controller.dispose_
      ;()super.dispose
      {

      override@

```

```

        } Widget build(BuildContext context)
            )return Scaffold
        ,backgroundColor: Theme.of(context).colorScheme.primary
            )body: Center
                )child: AnimatedBuilder
                    ,animation: _controller
                } builder: (context, child)
                    )return FadeTransition
                ,opacity: _fadeAnimation
                    )child: ScaleTransition
                ,scale: _scaleAnimation
                    )child: Column
        ,mainAxisAlignment: MainAxisAlignment.center
            ] :children
                App Icon //
                )Container
                ,width: 120
                ,height: 120
            )decoration: BoxDecoration
                ,color: Colors.white
        ,borderRadius: BorderRadius.circular(24)
            ] :boxShadow
            )BoxShadow
        ,color: Colors.black.withOpacity(0.2)
            ,blurRadius: 10
        ,offset: const Offset(0, 5)
            ,(
            ,[
            ,(
        )child: const Icon
            ,Icons.eco
            ,size: 60
        ,color: Color(0xFF2E7D32)
            ,(
            ,(
            ...
        ,const SizedBox(height: 24)

        App Title //
        )Text

```

```

        , 'זיהוי אתרוגים',
        style: Theme.of(context).textTheme.headlineMedium?.copyWith
            ,color: Colors.white
            ,fontWeight: FontWeight.bold
            ,(
            ,(

        ,const SizedBox(height: 8)

    )Text
    , 'בינה מלאכותית לזיהוי כשרות',
    style: Theme.of(context).text
    ...

```

```

        lib/core/app_config.dart //
        ;'import 'dart:io
        ;'import 'package:flutter/services.dart
        ;'import 'package:path_provider/path_provider.dart

    } class AppConfig
;static const String modelFileName = 'etrog_classifier_v1.tflite
;static const String labelsFileName = 'etrog_labels.txt
;static const double modelInputSize = 224.0
;static const int modelBatchSize = 1
;static const int modelChannels = 3

        Performance settings //
;static const int cameraFPS = 30
;static const double confidenceThreshold = 0.7
static const int maxInferenceTime = 500; // milliseconds

        App settings //
;static const String appVersion = '1.0.0
;static const bool debugMode = true

;static late String modelPath
;static late String labelsPath
;static late List<String> labels

    } static Future<void> initialize() async
    } try

```

```

        Copy model from assets to local storage //
;()final appDir = await getApplicationDocumentsDirectory
;'modelPath = '${appDir.path}/${modelFileName}
;'labelsPath = '${appDir.path}/${labelsFileName}

        ...

        Copy model file //
;final modelData = await rootBundle.load('assets/models/$modelFileName')
;'final modelFile = File(modelPath)
;await modelFile.writeAsBytes(modelData.buffer.asUint8List())

        Copy and load labels //
;final labelsData = await rootBundle.loadString('assets/models/$labelsFileName')
;'final labelsFile = File(labelsPath)
;await labelsFile.writeAsString(labelsData)

        labels = labelsData.split('\n')
        where((line) => line.trim().isEmpty).
;'()toList.

;print('AppConfig initialized successfully')
;'print('Model path: $modelPath')
;'print('Labels count: ${labels.length}')

        } catch (e) {
;print('Error initializing AppConfig: $e')
;'rethrow
        {
        ...

        {
        {

        lib/core/theme/app_theme.dart //
;'import 'package:flutter/material.dart

        } class AppTheme
        Color palette //
        static const Color primaryColor = Color(0xFF2E7D32); // Green
        static const Color secondaryColor = Color(0xFFFF8F00); // Orange
;'static const Color backgroundColor = Color(0xFFF5F5F5)

```

```
        ;static const Color surfaceColor = Colors.white
        ;static const Color errorColor = Color(0xFFD32F2F)
;static const Color successColor = Color(0xFF388E3C)
;static const Color warningColor = Color(0xFFFF57C0)
```

```
                                Text colors //
        ;static const Color textPrimary = Color(0xFF212121)
;static const Color textSecondary = Color(0xFF757575)
        ;static const Color textOnPrimary = Colors.white
```

```
    } static ThemeData get lightTheme
        )return ThemeData
            ,useMaterial3: true
)colorScheme: ColorScheme.fromSeed
    ,seedColor: primaryColor
    ,brightness: Brightness.light
    ,primary: primaryColor
    ,secondary: secondaryColor
    ,surface: surfaceColor
    ,background: backgroundColor
    ,error: errorColor
    ,(
        ...
```

```
                                App Bar Theme //
)AppBarTheme: const AppBarTheme
    ,elevation: 0
    ,centerTitle: true
    ,backgroundColor: primaryColor
    ,foregroundColor: textOnPrimary
    )titleTextStyle: TextStyle
        ,fontSize: 20
    ,fontWeight: FontWeight.bold
    ,color: textOnPrimary
    ,fontFamily: 'Heebo'
    ,(
        ,(
```

```
                                Card Theme //
)cardTheme: CardTheme
    ,elevation: 4
```

```

        )shape: RoundedRectangleBorder
        ,borderRadius: BorderRadius.circular(12)
        ,
        ,margin: const EdgeInsets.all(8)
        ,
        Elevated Button Theme //
    )elevatedButtonTheme: ElevatedButtonThemeData
        )style: ElevatedButton.styleFrom
            ,elevation: 2
        ,padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12)
        )shape: RoundedRectangleBorder
        ,borderRadius: BorderRadius.circular(8)
        ,
        )textStyle: const TextStyle
            ,fontSize: 16
        ,fontWeight: FontWeight.w600
        ,fontFamily: 'Heebo
        ,
        ,
        ,
        Text Theme //
    )textTheme: const TextTheme
        )headlineLarge: TextStyle
            ,fontSize: 32
        ,fontWeight: FontWeight.bold
        ,color: textPrimary
        ,fontFamily: 'Heebo
        ,
        )headlineMedium: TextStyle
            ,fontSize: 24
        ,fontWeight: FontWeight.w600
        ,color: textPrimary
        ,fontFamily: 'Heebo
        ,
        )titleLarge: TextStyle
            ,fontSize: 20
        ,fontWeight: FontWeight.w500
        ,color: textPrimary
        ,fontFamily: 'Heebo

```

```

        ),
        )bodyLarge: TextStyle
            ,fontSize: 16
            ,color: textPrimary
            ,fontFamily: 'Heebo
        ),
        )bodyMedium: TextStyle
            ,fontSize: 14
            ,color: textSecondary
            ,fontFamily: 'Heebo
        ),
        ),

        Input Decoration Theme //
    )inputDecorationTheme: InputDecorationTheme
        )border: OutlineInputBorder
            ,borderRadius: BorderRadius.circular(8)
        ),
        )focusedBorder: OutlineInputBorder
            ,borderRadius: BorderRadius.circular(8)
        ,borderSide: const BorderSide(color: primaryColor, width: 2)
        ),
        ),
        ;
    ...

    {

    } static ThemeData get darkTheme
        )return ThemeData
            ,useMaterial3: true
        )colorScheme: ColorScheme.fromSeed
            ,seedColor: primaryColor
            ,brightness: Brightness.dark
        ),
        Add dark theme customizations here //
        ;
        {
        {
        {

lib/providers/app_state_provider.dart //

```

```

;import 'package:flutter/material.dart

} class AppStateProvider extends ChangeNotifier
    ;Locale _currentLocale = const Locale('he', 'IL')
;ThemeMode _themeMode = ThemeMode.system
    ;bool _isInitialized = false
    ;String? _lastError

    Getters //
    ;Locale get currentLocale => _currentLocale
;ThemeMode get themeMode => _themeMode
    ;bool get isInitialized => _isInitialized
    ;String? get lastError => _lastError

    Methods //
    } void setLocale(Locale locale)
    } if (_currentLocale != locale)
        ;currentLocale = locale_
        ;()notifyListeners
        {
        {

    } void setThemeMode(ThemeMode mode)
        } if (_themeMode != mode)
            ;themeMode = mode_
            ;()notifyListeners
            {
            {

    } void setInitialized(bool initialized)
        } if (_isInitialized != initialized)
            ;isInitialized = initialized_
            ;()notifyListeners
            {
            {

    } void setError(String? error)
        ;lastError = error_
        ;()notifyListeners
        {

```

```

        } ()void clearError
        ;lastError = null_
        ;()notifyListeners
        {
        {

lib/providers/ml_provider.dart //
        ;'import 'dart:io
        ;'import 'dart:typed_data
        ;'import 'package:flutter/material.dart
; 'import 'package:tflite_flutter/tflite_flutter.dart
        ;import 'package:image/image.dart' as img

        ;'import '../core/app_config.dart
        ;'import '../models/etrog_result.dart
        ;'import '../services/image_processor.dart

    } class MLProvider extends ChangeNotifier
        ;Interpreter? _interpreter
        ;bool _isModelLoaded = false
        ;bool _isProcessing = false
        ;String? _lastError

        Getters //
;bool get isModelLoaded => _isModelLoaded
        ;bool get isProcessing => _isProcessing
        ;String? get lastError => _lastError

        Initialize TensorFlow Lite model //
    } Future<void> initializeModel() async
        } try
        ;setProcessing(true)_
        ;()clearError_

        ...

        Load the model //
;interpreter = await Interpreter.fromFile(File(AppConfig.modelPath))_

        Allocate tensors //
        ;()interpreter!.allocateTensors_

```

```

        Verify model input/output shapes //
        ;final inputShape = _interpreter!.getInputTensor(0).shape
        ;final outputShape = _interpreter!.getOutputTensor(0).shape

        ;print('Model loaded successfully')
        ;print('Input shape: $inputShape')
        ;print('Output shape: $outputShape')

        ;isModelLoaded = true_

        } catch (e) {
        ;setError('Failed to load model: $e')_
        ;print('Error loading model: $e')
        } finally {
        ;setProcessing(false)_
        {
        ...

        {

        Classify etrog from image //
    } Future<EtrogResult?> classifyImage(String imagePath) async
        } if (!_isModelLoaded || _interpreter == null)
            ;setError('Model not loaded')_
            ;return null
            {
            ...
            } try
            ;setProcessing(true)_
            ;()clearError_

        ;()final stopwatch = Stopwatch()××start

        Process image //
        ;final processedImage = await ImageProcessor.preprocessImage(imagePath)
        } if (processedImage == null)
        ;setError('Failed to process image')_
        ;return null
        {

```

```

        Prepare input tensor //
        ;final input = _prepareInputTensor(processedImage)
        final output = List×filled(AppConfig×labels×length, 0.0)
        ;reshape([1, AppConfig.labels.length]).

        Run inference //
        ;interpreter!.run(input, output)_

        ;()stopwatch.stop

        Process output //
        ;final result = _processOutput(output[0], stopwatch.elapsedMilliseconds)

        ;print('Inference completed in ${stopwatch.elapsedMilliseconds}ms')
        ;return result

    } catch (e) {
        ;setError('Classification failed: $e')_
        ;print('Error during classification: $e')
        ;return null
    } finally {
        ;setProcessing(false)_
        {
            ...

        }

        Prepare input tensor from processed image //
    } List<List<List<List<double>>>> *prepareInputTensor(img.Image image)
        )final input = List.generate
            ,1
            )List.generate <= (*)
            ,()AppConfig.modelInputSize.toInt
            )List.generate <= (*)
            ,()AppConfig.modelInputSize.toInt
            ,List.filled(3, 0.0) <= (*)
            ,(
            ,(
            ;(
            ...

```

```

        } for (int y = 0; y < image.height; y++)
        } for (int x = 0; x < image.width; x++)
            ;final pixel = image.getPixel(x, y)
        input[0][y][x][0] = pixel.r / 255.0; // Red
        input[0][y][x][1] = pixel.g / 255.0; // Green
        input[0][y][x][2] = pixel.b / 255.0; // Blue
    }
    {

        ;return input
        ...

    }

    Process model output to EtrogResult //
} EtrogResult _processOutput(List<double> output, int inferenceTime)
    Find the class with highest confidence //
        ;double maxConfidence = 0.0
        ;int maxIndex = 0

        ...

    } for (int i = 0; i < output.length; i++)
        } if (output[i] > maxConfidence)
            ;maxConfidence = output[i]
            ;maxIndex = i
        {
        {

        Create confidence map //
        ;{}<final confidenceMap = <String, double
    } for (int i = 0; i < AppConfig.labels.length && i < output.length; i++)
        ;confidenceMap[AppConfig.labels[i]] = output[i]
        {

        Determine result status //
        ;EtrogStatus status
    } if (maxConfidence < AppConfig.confidenceThreshold)
        ;status = EtrogStatus.uncertain
        } else {
;()final label = AppConfig.labels[maxIndex].toLowerCase
    } (label.contains('kosher') || ('כשר')label.contains) if

```

```

        ;status = EtrogStatus.kosher
    } (label.contains('invalid') || ('לִיטוּ')label.contains) else if {
        ;status = EtrogStatus.invalid
    } else {
        ;status = EtrogStatus.uncertain
    }
    {
    {

        )return EtrogResult
        ,predictedClass: AppConfig.labels[maxIndex]
        ,confidence: maxConfidence
        ,status: status
        ,confidenceMap: confidenceMap
        ,inferenceTime: inferenceTime
        ,()timestamp: DateTime.now
        ;(
        ...

    {

```

```

        Helper methods //
    } void _setProcessing(bool processing)
    } if (_isProcessing != processing)
        ;isProcessing = processing_
        ;()notifyListeners
    {
    {

```

```

    } void _setError(String error)
        ;lastError = error_
        ;()notifyListeners
    {

```

```

    } ()void _clearError
    } if (_lastError != null)
        ;lastError = null_
        ;()notifyListeners
    {
    {

```

override@

```

        } ()void dispose
;()interpreter?.close_
;()super.dispose
        {
        {

lib/models/etrog_result.dart //
    } enum EtrogStatus
        ,kosher
        ,invalid
        ,uncertain
        {

    } class EtrogResult
;final String predictedClass
;final double confidence
;final EtrogStatus status
;final Map<String, double> confidenceMap
;final int inferenceTime
;final DateTime timestamp
;final String? variety
;final List<String>? detectedDefects
;final Map<String, dynamic>? additionalInfo

    })const EtrogResult
,required this.predictedClass
,required this.confidence
,required this.status
,required this.confidenceMap
,required this.inferenceTime
,required this.timestamp
,required this.variety
,required this.detectedDefects
,required this.additionalInfo
;({

    Helper getters //
;bool get isKosher => status == EtrogStatus.kosher
;bool get isInvalid => status == EtrogStatus.invalid
;bool get isUncertain => status == EtrogStatus.uncertain

```

```

    } String get statusText
    } switch (status)
    :case EtrogStatus.kosher
        ;'נשר' return
    :case EtrogStatus.invalid
        ;'פסול' return
    :case EtrogStatus.uncertain
        ;'לא ברור' return
    {
    {

```

```

    } Color get statusColor
    } switch (status)
    :case EtrogStatus.kosher
return const Color(0xFF4CAF50); // Green
    :case EtrogStatus.invalid
return const Color(0xFFFF44336); // Red
    :case EtrogStatus.uncertain
return const Color(0xFFFF9800); // Orange
    {
    {

```

```

;'%String get confidencePercentage => '${(confidence * 100).toStringAsFixed(1)}

```

```

        Convert to JSON for storage //
    } ()Map<String, dynamic> toJson
        } return
        ,predictedClass': predictedClass'
        ,confidence': confidence'
        ,()status': status.toString'
        ,confidenceMap': confidenceMap'
        ,inferenceTime': inferenceTime'
        ,()timestamp': timestamp.toIso8601String'
        ,variety': variety'
        ,detectedDefects': detectedDefects'
        ,additionalInfo': additionalInfo'
        ;{
        {

```

```

        Create from JSON //
    } factory EtrogResult.fromJson(Map<String, dynamic> json)

```

```

        )return EtrogResult
        ,predictedClass: json['predictedClass']
        ,confidence: json['confidence']
        )status: EtrogStatus.values.firstWhere
        ,extoString() == json['status'] <= (e)
        ,(
        ,confidenceMap: Map<String, double>.from(json['confidenceMap'])
        ,inferenceTime: json['inferenceTime']
        ,timestamp: DateTime.parse(json['timestamp'])
        ,variety: json['variety']
        ,()<detectedDefects: json['detectedDefects']?.cast<String>
        ,additionalInfo: json['additionalInfo']
        ;(
        {
        {

lib/services/image_processor.dart //
    ;'import 'dart:io
;import 'package:image/image.dart' as img
    ;'import '../core/app_config.dart

    } class ImageProcessor
    Preprocess image for model input //
} static Future<img.Image?> preprocessImage(String imagePath) async
    } try
        Read image file //
        ;final imageFile = File(imagePath)
        ;()final imageBytes = await imageFile.readAsBytes

        ...

        Decode image //
;img.Image? image = img.decodeImage(imageBytes)
        ;if (image == null) return null

        Apply preprocessing pipeline //
        ;image = _resizeImage(image)
;image = _normalizeImage(image)
        ;image = _enhanceImage(image)

        ;return image
        } catch (e) {

```

```

        ;print('Error preprocessing image: $e')
        ;return null
    }
    ...

    {

        Resize image to model input size //
    } static img.Image _resizeImage(img.Image image)
        ;()final size = AppConfig×modelInputSize×toInt
        )return img.copyResize
            ,image
            ,width: size
            ,height: size
        ,interpolation: img.Interpolation.cubic
        ;(
        {

            Normalize image values //
        } static img.Image _normalizeImage(img.Image image)
            Apply histogram equalization for better contrast //
        )return img.adjustColor
            ,image
            ,contrast: 1.1
            ,brightness: 1.05
        ;(
        {

            Enhance image quality //
        } static img.Image _enhanceImage(img.Image image)
            Apply slight sharpening //
        )return img.convolution
            ,image
            ] :filter
            ,0 ,1- ,0
            ,1- ,5 ,1-
            ,0 ,1- ,0
            ,[
            ,div: 1
            ;(
            {

```

```

        Extract color features for additional analysis //
    } static Map<String, double> extractColorFeatures(img.Image image)
        ;int totalPixels = image.width * image.height
        ;double totalR = 0, totalG = 0, totalB = 0
        ;double totalH = 0, totalS = 0, totalV = 0

        ...

        } for (int y = 0; y < image.height; y++)
        } for (int x = 0; x < image.width; x++)
            ;final pixel = image.getPixel(x, y)

            RGB values //
            ;totalR += pixel.r
            ;totalG += pixel.g
            ;totalB += pixel.b

            Convert to HSV //
            ;final hsv = _rgbToHsv(pixel.r, pixel.g, pixel.b)
            ;!totalH += hsv['h']
            ;!totalS += hsv['s']
            ;!totalV += hsv['v']
            {
            {

            } return
            ,avgR': totalR / totalPixels / 255.0'
            ,avgG': totalG / totalPixels / 255.0'
            ,avgB': totalB / totalPixels / 255.0'
            ,avgH': totalH / totalPixels'
            ,avgS': totalS / totalPixels'
            ,avgV': totalV / totalPixels'
            ;{
            ...

            {

            Convert RGB to HSV //
        } static Map<String, double> _rgbToHsv(num r, num g, num b)
            ;r = r / 255.0
            ;g = g / 255.0

```

```

        ;b = b / 255.0
        ```
;final max = [r, g, b].reduce((a, b) => a > b ? a : b)
;final min = [r, g, b].reduce((a, b) => a < b ? a : b)
;final diff = max - min

 ;double h = 0
 } if (diff != 0)
 } if (max == r)
;h = (60 * ((g - b) / diff) + 360) % 360
 } else if (max == g) {
;h = (60 * ((b - r) / diff) + 120) % 360
 } else if (max == b) {
;h = (60 * ((r - g) / diff) + 240) % 360
 {
 {

;final s = max == 0 ? 0 : diff / max
;final v = max

;return {'h': h, 's': s, 'v': v}
        ```

        {
        {

```